

Assignment-6

```

1) #include <stdio.h>
    struct process {
        int pid;
        int arrival;
        int burst;
        int priority;
        int waiting;
        int turnaround;
        int completion;
        int remaining;
    };

    void sort(struct process p[], int n) {
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - 1 - i; j++) {
                if (p[j].arrival > p[j+1].arrival) {
                    struct process temp = p[j];
                    p[j] = p[j+1];
                    p[j+1] = temp;
                }
            }
        }
    }

    void fcts(struct process p[], int n) {
        sort(p, n);
        int time = 0;
        for (int i = 0; i < n; i++) {
            if (time < p[i].arrival) {
                time = p[i].arrival;
            }
            time += p[i].burst;
            p[i].completion = time;
            p[i].turnaround = p[i].completion - p[i].arrival;
        }
    }

```


$$p[i].waiting = p[i].turnaround - p[i].burst;$$

}

}

```
void sjf(struct process p[], int n) {
```

```
    sort(p, n);
```

```
    int time = 0;
```

```
    int done = 0; minIndex;
```

```
    int completed[n];
```

```
    for(int i=0; i<n; i++) completed[i] = 0;
```

```
    while(done != n) {
```

```
        minIndex = -1;
```

```
        for(int i=0; i<n; i++) {
```

```
            if (p[i].arrival <= time && !completed[i]) {
```

```
                if (minIndex == -1 || p[i].burst <
```

```
                    p[minIndex])
```

```
                    minIndex = i;
```

}

}

```
    if (minIndex == -1) {
```

```
        time++;
```

```
    } else {
```

```
        time += p[minIndex].burst;
```

```
        p[minIndex].completion = time;
```

```
        completed[minIndex] = 1;
```

```
        done++;
```

}

}

Name: Avancha Kiran Kumar

Regno: 20233093

CSE A-2

```
void rr(struct process p[], int n, int q) {
    int time = 0, done = 0;
    for(int i = 0; i < n; i++) {
        p[i].remaining = p[i].burst;
        while(done != n) {
            for(int i = 0; i < n; i++) {
                if(p[i].remaining > 0) {
                    time += 1;
                    p[i].remaining -= q;
                } else {
                    time += p[i].remaining;
                    p[i].completion = time;
                    p[i].turnaround = p[i].completion - p[i].arrival;
                    p[i].remaining = 0;
                    done++;
                }
            }
        }
    }
}
```

```
void ps(struct process p[], int n) {
    int done = 0, time = 0, high;
    int completed[n];
    for(int i = 0; i < n; i++) {
        completed[i] = 0;
        while(done != n) {
            for(int i = 0; i < n; i++) {
                if(p[i].arrival <= time && !completed[i]) {
                    if(high == -1 || p[i].priority > p[high]) {
                        high = i;
                    }
                }
            }
            if(high != -1) {
                time += p[high].remaining;
                p[high].completion = time;
                p[high].turnaround = p[high].completion - p[high].arrival;
                p[high].remaining = 0;
                completed[high] = 1;
                done++;
            }
        }
    }
}
```


Name :- Avancha Kisan Kumar

Regno :- 20233093

CSE A-2

```
if (highindex == -1) {
```

```
    time++;
```

```
} else {
```

```
    Time += p[highindex].burst;
```

```
    p[highindex].completion = time;
```

```
    p[highindex].turnaround = p[highindex].completion -
```

```
        p[highindex].arrival;
```

```
    p[highindex].waiting = p[highindex].turnaround -
```

```
        p[highindex].burst;
```

```
    completed[highindex] = 1;
```

```
    done++;
```

```
}
```

```
}
```

```
void print(struct process p[], int n, const char method[]) {
```

```
    printf(" %n %s scheduling : \n", method);
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf(" p %d | t %d | t %d | t %d | t %d |",
```

```
            p[i].pid, p[i].arrival, p[i].burst, p[i].completion,
```

```
            p[i].waiting, p[i].turnaround)
```

```
    }
```

```
int main() {
```

```
    struct process processes[] = { {1, 0, 2, 4, 1, 0, 0, 0},
```

```
        {2, 2, 3, 4, 0, 0, 0, 0},
```

```
        {3, 1, 3, 3, 0, 0, 0, 0},
```

```
        {4, 3, 7, 2, 0, 0, 0, 0} }
```

```
}
```

```
int n = 4;
```

```
struct process p1[n], p2[n], p3[n], p4[n];
```

```
for (int i = 0; i < n; i++)
```

```
    p1[i] = p2[i] = p3[i] = processes[i];
```

```
}
```


Name: Avancha Kiran Kumar

RegNo: 20233093

CSE A-2

```
fcfs(p1, n);
```

```
print(p1, n, "FLFS");
```

```
sjf(p2, n);
```

```
print(p2, n, "SJF");
```

```
print(p3, n, "1 Rad Rottn");
```

```
ps(p4, n);
```

```
print(p4, n, "priority");
```

```
return 0;
```

```
}
```

2)

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAX_PS
```

```
typedef struct
```

```
{
```

```
    char id;
```

```
    int at, bt, rt, ct, wt, tat;
```

```
};
```

```
process;
```

```
int fsf(process p[], int n, int time) {
```

```
    int minindex = 1, min_rt = INT_MAX;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (p[i].at <= time && p[i].rt > 0 &&
```

```
            p[i].rt < min_rt) {
```

```
            min_rt = p[i].rt;
```

```
            min_index = i;
```

```
        }
```

```
    }
```

```
    }
```

```
    return min_index;
```

```
}
```


$$\{1, 5, 2, 2, 0, 0, 0\}$$

Name: Avamche Kiran Kumar

Regno: 20233093

CSE A-2

```
sortf(p-mem-p);  
return 0;  
}
```

```
3) #include <stdio.h>  
#include <string.h>  
#define MEM-P 4  
typedef struct {  
    char id[33];  
    int at, bt, ct, wt, tat;  
    char type[103];  
} process;  
void calc-times(process p[3], int n) {  
    int total-wt=0, total-tat=0, time=0;  
    for (int i=0; i<n; i++) {  
        if (time < p[i].at) {  
            time = p[i].at;  
            p[i].ct = time + p[i].bt;  
            p[i].tat = p[i].ct - p[i].at;  
            p[i].wt = p[i].tat - p[i].bt;  
            time = p[i].ct;  
            total-wt += p[i].wt;  
            total-tat += p[i].tat;  
        }  
    }  
    for (int i=0; i<n; i++) {  
        p[i].id, p[i].type, p[i].at, p[i].bt, p[i].ct,  
        p[i].tat, p[i].wt;  
    }  
}
```


Name: Avancha Kiran Kumar

Regno: 20233093

CSE A2

Page :

Date : / /

```
void mlg_sch (process p[], int n) {
    process sys = q[Max-p], user = q[Max-p];
    int sys_c = 0, user_c = 0;
    for (int i = 0; i < n; i++) {
        if (strcmp(p[i].type, "system") == 0)
            else
                user_c = (user_c + 1) = p[i];
    }
    Calc_times (user_q, user_c);
    Calc_times (sys_q, sys_c);
    printf ("In executing system processes (High priority)");
}
```

```
int main() {
    process p[Max-p] = {{"p1", 0, 3, 0, 0, 0, "User"},
                        {"p2", 1, 2, 0, 0, 0, "system"},
                        {"p3", 2, 5, 0, 0, 0, "User"},
                        {"p4", 3, 1, 0, 0, 0, "system"}
    };
    printf ("Multi-level Queue scheduling (Fixed priority)\n");
    mlg_sch (p, Max-p);
    return 0;
}
```